

Database Design — GymTracker

Overview

GymTracker uses a **SQLite** relational database managed through **Flask-SQLAlchemy**. The schema consists of four tables that model users, exercises, workouts, and the many-to-many relationship between workouts and exercises.

Table Definitions

1. users

Stores registered users of the application.

Column	Type	Constraints
user_id	INTEGER	PRIMARY KEY, AUTO-INCREMENT
name	VARCHAR(120)	NOT NULL
email	VARCHAR(200)	NOT NULL, UNIQUE

- **Cascade behavior:** Deleting a user deletes all their workouts (and transitively, all associated workout_exercises).
-

2. exercises

Reference table of available exercises, grouped by muscle group.

Column	Type	Constraints
exercise_id	INTEGER	PRIMARY KEY, AUTO-INCREMENT
name	VARCHAR(200)	NOT NULL
muscle_group	VARCHAR(100)	NOT NULL

Indexes:

- ix_exercises_muscle_group on muscle_group — speeds up filtering exercises by muscle group.
-

3. workouts

Represents a single workout session for a user on a given date.

Column	Type	Constraints
workout_id	INTEGER	PRIMARY KEY, AUTO-INCREMENT
user_id	INTEGER	NOT NULL, FOREIGN KEY → users.user_id
date	DATE	NOT NULL
notes	TEXT	DEFAULT ""

Indexes:

- ix_workouts_user_id on user_id — speeds up user-specific workout lookups.

- `ix_workouts_date` on `date` — speeds up date-range filtering in reports.

Cascade behavior: Deleting a workout deletes all associated `workout_exercises`.

4. workout_exercises

Junction table linking workouts to exercises, with set/rep/weight data for each entry.

Column	Type	Constraints
<code>id</code>	INTEGER	PRIMARY KEY, AUTO-INCREMENT
<code>workout_id</code>	INTEGER NOT NULL	FOREIGN KEY → <code>workouts.workout_id</code>
<code>exercise_id</code>	INTEGER NOT NULL	FOREIGN KEY → <code>exercises.exercise_id</code>
<code>sets</code>	INTEGER NOT NULL	
<code>reps</code>	INTEGER NOT NULL	
<code>weight</code>	FLOAT NOT NULL	(in lbs)

Indexes:

- `ix_workout_exercises_workout_id` on `workout_id`
- `ix_workout_exercises_exercise_id` on `exercise_id`

Constraints Summary

Constraint Type	Details
Primary Keys	<code>user_id</code> , <code>exercise_id</code> , <code>workout_id</code> , <code>id</code> (all auto-increment)
Foreign Keys	<code>workouts.user_id</code> → <code>users.user_id</code> ; <code>workout_exercises.workout_id</code> → <code>workouts.workout_id</code> ; <code>workout_exercises.exercise_id</code> → <code>exercises.exercise_id</code>
Unique	<code>users.email</code>
Not Null	All columns except <code>workouts.notes</code>
Cascade Delete	<code>users</code> → <code>workouts</code> → <code>workout_exercises</code> (delete-orphan)

Indexes Summary

Index Name	Table	Column(s)	Purpose
<code>ix_exercises_muscle_group</code>	<code>exercises</code>	<code>muscle_group</code>	Filter exercises by muscle group
<code>ix_workouts_user_id</code>	<code>workouts</code>	<code>user_id</code>	User-specific workout queries
<code>ix_workouts_date</code>	<code>workouts</code>	<code>date</code>	Date-range filtering in reports
<code>ix_workout_exercises_workout_id</code>	<code>workout_exercises</code>	<code>workout_id</code>	Join performance
<code>ix_workout_exercises_exercise_id</code>	<code>workout_exercises</code>	<code>exercise_id</code>	Join performance

Seed Data

The database is seeded on first run with:

- **16 exercises** across 6 muscle groups (Chest, Back, Legs, Shoulders, Arms, Core)
 - **2 sample users** (Alice Johnson, Bob Smith)
 - **4 sample workouts** with associated workout_exercises
-

Design Decisions

1. **Separate exercises reference table:** Exercises are stored independently so they can be reused across workouts and users. This avoids duplication and enables consistent muscle-group filtering.
2. **Junction table with attributes:** workout_exercises is more than a simple join table — it carries sets, reps, and weight, which vary per workout. This models the real-world concept of performing the same exercise with different parameters on different days.
3. **Cascade deletes:** Configured at the ORM level (cascade="all, delete-orphan") to maintain referential integrity. Deleting a user cleans up all their data automatically.
4. **Strategic indexing:** Indexes are placed on foreign key columns and columns frequently used in WHERE/JOIN clauses (date, muscle_group) to optimize the report query, which joins all four tables with multiple optional filters.